PYTHON PARA SOC N1

Volume 1 - Fundamentos e Análise de Logs

Caderno prático para iniciantes

Aprenda Python resolvendo tarefas parecidas com as de um analista SOC: leitura de logs, extração de campos, contagem de eventos, identificação de IPs e investigação de autenticações.

Regra do caderno: tente cada exercício antes de consultar as soluções. O objetivo não é decorar código; é aprender a transformar uma pergunta de segurança em pequenos passos lógicos.

Nível: iniciante → SOC N1

Idioma: Português

Como usar este caderno

Faça os exercícios no seu editor ou terminal. Crie uma pasta chamada python_soc_n1 e salve cada exercício em um arquivo separado, por exemplo ex01.py, ex02.py etc.

Símbolo	Significado
★	Fundamento de Python
★★	Python aplicado a logs
★★★	Raciocínio de investigação SOC

Importante: os logs deste material são fictícios e foram criados apenas para estudo.

Módulo 1 - O Python que um SOC iniciante precisa

Você não precisa dominar toda a linguagem para começar. Neste volume vamos usar principalmente variáveis, strings, condições, loops, listas, dicionários, arquivos e algumas operações de texto.

1.1 Variáveis e strings

```
evento = "Failed password"
ip = "192.168.1.50"
tentativas = 7

print(evento)
print(ip)
print(tentativas)
```

Uma variável guarda um valor. Em logs, esses valores podem representar um IP, usuário, tipo de evento, horário ou contador.

Exercício 1 - Primeiro alerta ★

Crie três variáveis: usuario, ip e evento. Coloque valores fictícios e mostre uma frase como: Usuário admin teve Failed password vindo de 10.0.0.5.

Exercício 2 - Classificando tentativas ★

Crie a variável tentativas = 8. Se o valor for maior que 5, mostre ALERTA: muitas tentativas. Caso contrário, mostre Quantidade normal.

> Dica: Você precisará de if e else.

1.2 Listas e loops

```
ips = ["10.0.0.5", "10.0.0.8", "192.168.1.20"]

for ip in ips:
    print(ip)
```

O for permite repetir uma ação. Isso é essencial porque um arquivo de log pode ter milhares de linhas.

Exercício 3 - Lista de IOCs ★

Crie uma lista com cinco endereços IP. Use for para imprimir um IP por linha.

Exercício 4 - Procurando um IP ★

Na mesma lista, mostre IP suspeito encontrado somente quando o IP for igual a 185.220.101.4.

1.3 Strings e split()

```
linha = "Failed password for admin from 203.0.113.50 port 55221"
pedacos = linha.split()

print(pedacos)
print(pedacos[5])
```

split() corta uma string em partes. Sem argumento, ele normalmente separa pelos espaços. Depois você acessa cada parte pelo índice: 0, 1, 2...

Exercício 5 - Separando um log ★

Use split() na linha abaixo e imprima cada pedaço separadamente.

```
Oct 07 10:14:32 server sshd Failed password for root from 203.0.113.50
```

Exercício 6 - Extraindo o IP ★★

Usando a mesma linha, descubra em qual posição está 203.0.113.50 e mostre somente o IP.

Exercício 7 - Extraindo usuário e IP ★★

A partir de Failed password for admin from 198.51.100.25, mostre a saída:
Usuário: admin
IP: 198.51.100.25.

Módulo 2 - Leitura de arquivos de log

No SOC, os dados quase nunca estarão escritos diretamente dentro do seu programa. Eles virão de arquivos, APIs, SIEMs ou outras fontes. Vamos começar por arquivos.

2.1 open(), with e readline()

```
with open("auth.log") as arquivo:
    linha = arquivo.readline()
    print(linha)
```

O with open(...) abre o arquivo e fecha automaticamente quando o bloco termina. readline() lê uma linha.

Exercício 8 - Lendo a primeira linha ★

Crie um arquivo auth.log com três linhas de texto. Faça um programa que leia e mostre somente a primeira linha.

Exercício 9 - Lendo o arquivo inteiro ★

Agora use um for para mostrar todas as linhas de auth.log.

Dica: Um arquivo aberto pode ser percorrido diretamente: for linha in arquivo.

2.2 Primeiro auth.log

```
Oct 07 10:10:01 web01 sshd[1201]: Failed password for root from 203.0.113.50 port 51111 ssh2
Oct 07 10:10:04 web01 sshd[1202]: Failed password for root from 203.0.113.50 port 51114 ssh2
Oct 07 10:10:10 web01 sshd[1203]: Accepted password for tales from 10.0.0.20 port 49200 ssh2
Oct 07 10:11:22 web01 sshd[1204]: Failed password for admin from 198.51.100.25 port 53001 ssh2
Oct 07 10:12:02 web01 sshd[1205]: Failed password for root from 203.0.113.50 port 51190 ssh2
```

Exercício 10 - Filtrando falhas ★★

Copie o log acima para auth.log. Mostre apenas as linhas que contêm Failed password.

Dica: Você pode testar: if "Failed password" in linha.

Exercício 11 - Filtrando sucessos ★★

Mostre somente as linhas que contêm Accepted password.

Exercício 12 - Contador de falhas ★★

Conte quantas linhas contêm Failed password e mostre no final: Total de falhas: X.

Exercício 13 - Falhas para root ★★

Conte somente as tentativas de login malsucedidas contra o usuário root.

Módulo 3 - Extração e contagem de IPs

Encontrar um evento é apenas o começo. Um analista normalmente precisa responder: quem fez isso?, quantas vezes? e houve sucesso depois?.

3.1 Dicionários como contadores

```
contagem = {}

ip = "203.0.113.50"

if ip not in contagem:
    contagem[ip] = 0

contagem[ip] += 1
print(contagem)
```

Um dicionário associa uma chave a um valor. Para SOC, uma aplicação muito útil é IP → número de eventos.

Exercício 14 - Extraia todos os IPs ★★

No auth.log anterior, para cada linha com Failed password, extraia e imprima somente o IP.

Dica: Procure a palavra "from". O IP aparece logo depois dela.

Exercício 15 - Conte por IP ★★

Crie um dicionário que conte quantas falhas vieram de cada IP.

Exercício 16 - Mostre o campeão ★★★

Mostre qual IP realizou mais tentativas malsucedidas. Não se preocupe ainda em fazer a solução mais elegante; faça funcionar.

Exercício 17 - Regra de alerta ★★★

Mostre somente IPs que tenham 3 ou mais tentativas malsucedidas.

3.2 Novo conjunto de dados

```
Oct 07 11:00:01 srv01 sshd: Failed password for root from 45.33.32.10 port 41001
Oct 07 11:00:03 srv01 sshd: Failed password for root from 45.33.32.10 port 41002
Oct 07 11:00:05 srv01 sshd: Failed password for admin from 185.10.10.8 port 50001
Oct 07 11:00:08 srv01 sshd: Failed password for root from 45.33.32.10 port 41003
Oct 07 11:00:11 srv01 sshd: Failed password for root from 45.33.32.10 port 41004
Oct 07 11:00:20 srv01 sshd: Accepted password for root from 45.33.32.10 port 41010
Oct 07 11:01:02 srv01 sshd: Accepted password for tales from 10.0.0.15 port 49001
```

Exercício 18 - Brute force seguido de sucesso ★★★

Descubra programaticamente se algum IP que gerou Failed password também aparece posteriormente em uma linha Accepted password. Mostre o IP.

Exercício 19 - Pequeno relatório ★★★

Gere uma saída contendo: total de falhas, IPs envolvidos, quantidade por IP e IPs que também tiveram autenticação aceita.

Módulo 4 - Parsing de audit.log

O Linux Audit registra eventos de segurança e sistema. As linhas têm muitos campos no formato chave=valor, por isso são ótimas para praticar parsing.

```
type=CONFIG_CHANGE msg=audit(1791187488.870:595): auid=1000 ses=3 subj=unconfined
type=USER_LOGIN msg=audit(1791187501.115:596): pid=2201 uid=0 auid=1000 ses=3 res=success
type=USER_LOGIN msg=audit(1791187522.901:597): pid=2210 uid=0 auid=1001 ses=4 res=failed
type=EXECVE msg=audit(1791187600.220:598): argc=3 a0="sudo" a1="systemctl" a2="stop"
type=USER_LOGIN msg=audit(1791187650.510:599): pid=2250 uid=0 auid=1001 ses=4 res=failed
```

4.1 Entendendo msg=audit(...)

Observe msg=audit(1791187488.870:595). Antes dos dois-pontos está o timestamp do evento; depois está o número serial do evento.

Exercício 20 - Extraindo type ★★

Da primeira linha, extraia somente CONFIG_CHANGE.

Exercício 21 - Timestamp e serial ★★

Da primeira linha, extraia:
Horário: 1791187488.870
Serial: 595.

Dica: Você pode resolver usando split() várias vezes.

Exercício 22 - Filtrando login ★★

Mostre somente os eventos cujo type seja USER_LOGIN.

Exercício 23 - Logins falhos ★★★

Conte quantos eventos USER_LOGIN possuem res=failed.

Exercício 24 - AUIDs envolvidos ★★★

Extraia os valores de auid dos logins falhos e mostre quais usuários numéricos aparecem.

Módulo 5 - Mini investigação SOC

Agora você recebe um pequeno incidente. Não existe uma única linha que dê a resposta. Você precisa correlacionar eventos.

Caso: servidor finance-web

```
Oct 07 02:14:01 finance-web sshd: Failed password for root from 198.51.100.77 port 60101
Oct 07 02:14:03 finance-web sshd: Failed password for root from 198.51.100.77 port 60102
Oct 07 02:14:05 finance-web sshd: Failed password for root from 198.51.100.77 port 60103
Oct 07 02:14:07 finance-web sshd: Failed password for root from 198.51.100.77 port 60104
Oct 07 02:14:09 finance-web sshd: Failed password for root from 198.51.100.77 port 60105
Oct 07 02:14:12 finance-web sshd: Accepted password for root from 198.51.100.77 port 60110
Oct 07 02:15:02 finance-web sudo: root : COMMAND=/usr/bin/cat /etc/shadow
Oct 07 02:16:41 finance-web sudo: root : COMMAND=/usr/bin/useradd backupsvc
Oct 07 02:17:10 finance-web sudo: root : COMMAND=/usr/bin/passwd backupsvc
Oct 07 08:02:11 finance-web sshd: Accepted password for tales from 10.0.0.15 port 50020
```

Cenário: o servidor normalmente recebe administração interna a partir da rede 10.0.0.0/24. O SOC recebeu um alerta de autenticação anormal durante a madrugada.

Exercício 25 - Triagem ★★

Conte o total de Failed password e Accepted password.

Exercício 26 - Origem suspeita ★★

Extraia os IPs das autenticações e identifique qual IP externo aparece repetidamente.

Exercício 27 - Correlação ★★★

Verifique se o IP com várias falhas conseguiu autenticar com sucesso.

Exercício 28 - Linha do tempo ★★★

Mostre, na ordem, somente os eventos relacionados ao usuário root.

Exercício 29 - Ações pós-login ★★★

Extraia e mostre os comandos executados via sudo após o login suspeito.

Exercício 30 - Conclusão SOC ★★★

Sem escrever código adicional, responda em 4 a 6 linhas: o que provavelmente aconteceu, qual IP é suspeito, qual conta foi comprometida e quais ações merecem investigação imediata.

Desafios extras - sem passo a passo

Estes exercícios servem para verificar se você consegue combinar o que aprendeu sem receber a técnica exata.

Exercício 31 - Top IPs

Leia um arquivo de autenticação e mostre os três IPs com maior número de falhas.

Exercício 32 - Usuários atacados

Conte quantas tentativas malsucedidas cada nome de usuário recebeu.

Exercício 33 - Alerta configurável

Crie uma variável limite = 5. O programa deve alertar qualquer IP cuja contagem seja igual ou superior ao limite.

Exercício 34 - Salvar relatório

Em vez de apenas imprimir o resultado, grave o relatório em relatorio_soc.txt.

Exercício 35 - Função de extração

Crie uma função extrair_ip(linha) que receba uma linha de autenticação e devolva o IP.

Exercício 36 - Função de análise

Crie uma função que receba o nome de um arquivo de log e devolva um dicionário com a contagem de falhas por IP.

Folha de consulta rápida

```
# Abrir e percorrer arquivo
with open("auth.log") as arquivo:
    for linha in arquivo:
        print(linha.strip())

# Procurar texto
if "Failed password" in linha:
    print(linha)

# Separar
pedacos = linha.split()

# Contador simples
total = 0
total += 1

# Dicionário contador
contagem = {}
if ip not in contagem:
    contagem[ip] = 0
contagem[ip] += 1

# Percorrer dicionário
for ip, quantidade in contagem.items():
    print(ip, quantidade)

# Escrever arquivo
with open("relatorio.txt", "w") as saida:
    saida.write("Relatório SOC\n")
```

Use esta página quando esquecer a sintaxe. Tente não copiar uma solução completa: consulte somente a peça de que precisa.

Soluções comentadas

Consulte esta seção somente depois de tentar. Existem várias formas corretas de resolver os exercícios; abaixo estão soluções simples, adequadas ao nível iniciante.

Exercícios 1 a 7

```
# Ex. 1
usuario = "admin"
ip = "10.0.0.5"
evento = "Failed password"
print("Usuário", usuario, "teve", evento, "vindo de", ip)

# Ex. 2
tentativas = 8
if tentativas > 5:
    print("ALERTA: muitas tentativas")
else:
    print("Quantidade normal")

# Ex. 3
ips = ["10.0.0.1", "10.0.0.2", "185.220.101.4", "192.168.1.5", "8.8.8.8"]
for ip in ips:
    print(ip)

# Ex. 4
for ip in ips:
    if ip == "185.220.101.4":
        print("IP suspeito encontrado")

# Ex. 5-7
linha = "Failed password for admin from 198.51.100.25"
pedacos = linha.split()
print(pedacos)
print("Usuário:", pedacos[3])
print("IP:", pedacos[5])
```

Exercícios 8 a 13

```
# Ex. 8
with open("auth.log") as arquivo:
    print(arquivo.readline())

# Ex. 9
with open("auth.log") as arquivo:
    for linha in arquivo:
        print(linha.strip())

# Ex. 10
with open("auth.log") as arquivo:
    for linha in arquivo:
        if "Failed password" in linha:
            print(linha.strip())

# Ex. 12
total = 0
with open("auth.log") as arquivo:
    for linha in arquivo:
        if "Failed password" in linha:
            total += 1
print("Total de falhas:", total)

# Ex. 13
total_root = 0
with open("auth.log") as arquivo:
```

```
    for linha in arquivo:
        if "Failed password for root" in linha:
            total_root += 1
print(total_root)
```

Exercícios 14 a 19

```
contagem = {}
ips_falha = set()
ips_sucesso = set()

with open("auth.log") as arquivo:
    for linha in arquivo:
        pedacos = linha.split()

        if "Failed password" in linha:
            pos = pedacos.index("from")
            ip = pedacos[pos + 1]
            ips_falha.add(ip)

            if ip not in contagem:
                contagem[ip] = 0
            contagem[ip] += 1

        if "Accepted password" in linha:
            pos = pedacos.index("from")
            ip = pedacos[pos + 1]
            ips_sucesso.add(ip)

print("Contagem:", contagem)

for ip, qtd in contagem.items():
    if qtd >= 3:
        print("ALERTA:", ip, qtd)

for ip in ips_falha:
    if ip in ips_sucesso:
        print("Falhou e depois autenticou:", ip)
```

Ideia importante: usamos pedacos.index('from') para localizar a palavra from. Assim, não dependemos tanto de um índice fixo.

Exercícios 20 a 24

```
linha = 'type=CONFIG_CHANGE msg=audit(1791187488.870:595): auid=1000 ses=3 subj=unconfined'
pedacos = linha.split()

# type
tipo = pedacos[0].split("=")[1]
print("Tipo:", tipo)

# msg=audit(1791187488.870:595):
msg = pedacos[1]
conteudo = msg.split("(")[1].split(")")[0]
horario = conteudo.split(":")[0]
serial = conteudo.split(":")[1]

print("Horário:", horario)
print("Serial:", serial)
```

Para os exercícios 22-24, aplique a mesma lógica dentro de um for linha in arquivo, filtre type=USER_LOGIN e depois teste res=failed.

Exercícios 25 a 30 - interpretação

Resultado esperado do caso: o IP 198.51.100.77 realizou cinco falhas contra root e depois obteve uma autenticação aceita. Logo depois, a conta root leu /etc/shadow, criou backupsvc e alterou sua senha. Isso é compatível com uma sequência de brute force/credential compromise seguida de ações pós-acesso e possível criação de persistência. O login de tales às 08:02 vem de um IP interno e, isoladamente, não apresenta o mesmo padrão.

Em um SOC real, isso seria o início da investigação, não a conclusão definitiva. Você procuraria mais evidências em outros logs, contexto do ativo e histórico do usuário.

Checklist de conclusão do Volume 1

☐ Consigo abrir e percorrer um arquivo com Python.

☐ Entendo a diferença entre readline() e percorrer o arquivo com for.

☐ Consigo usar split() para separar campos.

☐ Consigo filtrar linhas com if e in.

☐ Consigo contar eventos.

☐ Consigo usar um dicionário para contar eventos por IP.

☐ Consigo correlacionar falhas e sucessos de autenticação.

☐ Consigo extrair campos simples de audit.log.

☐ Consigo transformar uma sequência de logs em uma hipótese de incidente.

Se você consegue fazer a maior parte dos itens sem olhar as soluções, está pronto para o próximo volume: Regex, JSON/CSV, Suricata, IOCs, hashes e automação de triagem.